

ECE 216 - Laboratory Assignment #4

Name: Quang Nguyen Email: qnguy13@u.rochester.edu Date: 26th November, 2024

Section 1: Overview

This laboratory assignment focused on implementing control strategies for a DC motor using the Explorer 16/32 Development Board equipped with a PIC32MX795F512L microcontroller and use the 99:1 Pololu Metal Gearmotor 25Dx54L mm LP 12 V with 48 CPR with encoder feedback as an angle sensor. In order to drive the motor, we use the H-Bridge Motor Driver Circuit to control the direction and speed of the motor based on pulse-width modulation (PWM) signals set by the OC4 module from the microcontroller. A supply voltage source around 10V is also used in order to provide power for the motor to turn itself. The primary objectives were to design and program a proportional (P) controller and a proportional-integral (PI) controller to regulate the motor's angular position accurately to match the reference angle of 90 degree from the MATLAB script. Hence in order to switch between different controllers, use S3 with LED3 for control with potentiometer, S4 with LED4 for proportional control and S5 with LED5 for proportional-integral control. While displaying the reference angle 75o from MATLAB and current angle from the encoder on to the LCD display using the NT7603 LCD controller. Each controller aimed to minimize error between the motor's current and desired reference angular positions while providing stable and responsive motor behavior. Then be able to generate transient response plots from MATLAB and see the feedback controller behaviors. In addition, for this program, the board is configured to run a system clock and peripheral bus clock at 80 MHz. Timer 5 running at 25 Hz is used for data collection and UART communication with MATLAB script for plotting, while Timer 2 running at 50 Hz is used for setting the direction of motor matching the desired angle of 90 degree based on what type of controller is used.

Section 2: Design

Exercise E1: First create a new MPLAB X IDE project with the name "Laboratory4Quang.X". And in order to interface with the LCD display and show the results of our current angle recorded from the motor and desired reference angle 90 degree from the MATLAB script, we keep the file of lcd_display_driver.c from Lab 1 in the source files and lcd_display_driver.h in the header files to utilize the features of the LCD display. And I create the main.c file that contains the entire code for the project, include <xc.h>, <stdio.h>, <sys/attribs.h>, "lcd_display_driver.h", <math.h>.

Exercise E2: Within the program file main.c, for this exercise we want to implement and use the potentiometer to rotate the motor to match the desired angle 90 degree. We need to configure the PIC32MX795F512L on the board running system clock and peripheral bus at 80MHz. This frequency is defined using special function register values, with 8MHz fcrystal, 2 (0b010) for FPLLIDIV, 1 (0b000) for PLLDIV, 20 (0b101) for PLLMUL, and 1 (0b00) for PBDIV. As $F_{cpu} = (F_{crystal} \times PLLMUL) / (FPLLIDIV \times PLLDIV)$, and $F_{peripheral} = F_{cpu} / PBDIV$; which will result in 80 MHz for both given the config statements. In addition, I also defined the values for baud rate of 230400 and sys_freq and pb_freq of 80MHz and for the U2MODE_VALUE to be used for U2BRG as the UART baud rate function of the peripheral bus frequency, the high baud rate generator bit (M) (which is 16), and the desired baud rate (B) (which is 230400). And define global variables for reference and current angle for storing and updating new angles from the encoder. Also define S3, S4, S5 as 0, 1, 2 and initially set

global variable integer mode to 0 as S3 mode, which can be changed using Button pressed. Global time for the time not to reach above 5s for exactly 125 samples. Global variables prev_A and prev_B are used for states A and B before transition to next states for the state machine implemented. Also define global variables for number of counts to calculate the angle rotated of the encoder, proportional gain which can be set for proportional control, and integral gain and integral sum as parts of for proportional-integral control calculation.

- Function int read_potentiometer(void):

Looking at the Explorer 16/32 Schematics, the PIM pin for the potentiometer is 20, which corresponds to analog input channel AN2 from the PIC32MX795F512L (PIM) Sheet. Hence, we assign CH0SA to channel 2 input to read by the ADC. Then the Successive Approximation procedure is performed to find the best digital approximation of the analog input value. By first ADC sampling AD1CON1bits.SAMP = 1. Then, we need to wait for enough time, by using two calls _CP0_GET_COUNT() for difference of unsigned integers for the elapsed time of the core timer. The sampling time is 2000 ticks of the core timer. This corresponds to $2000 \times 25\text{ns} = 50$ microseconds of time. Then ADC starts the conversion, AD1CON1bits.SAMP = 0. Once the conversion is complete with checking flag AD1CON1bits.DONE, then the resulting digital value of the ADC process is stored in a buffer ADC1BUF0 to be used when the function is called.

- Function UART2 initialization:

This function initializes UART2 to be used, by setting U2MODEbits.BRGH to 0, which set UART2 to M = 16. Then U2BRG is equal to U2MODE_VALUE defined. The PDSEL bit on the U2MODE SFR to 0 sets 8-bit data and no parity. The STSEL bit to 0 sets 1 stop bit. The URXEN and UTXEN bits of the U2STA SFR to 1 to enable RX and TX pins. The UEN bit of the U2MODE SFR to 0 as there is no need for CTS and RTS in this program for hardware flow control. And finally U2MODEbits.ON to 1 to enable UART2.

- Function UART2 write string:

This sends a string of data over UART by iterating through each character of a provided string until it reaches the end (indicated by the null terminator, \0). It takes a const char* data as its argument, which points to the first character of the string. Inside the function, a “while” loop continues as long as the current character is not the null terminator, meaning there are characters left to send. Within the loop, another “while” statement checks the status of the U2STAbits.UTXBF flag, which indicates whether the UART transmit buffer (UTXBF) is full. If the buffer is full, the function waits until space becomes available, ensuring that no data is lost or overwritten. When space opens in the buffer, the character at the current data pointer location is transferred to the U2TXREG register, which sends it over UART. After each character is transmitted, the data pointer increments to the next character in the string. Once all characters have been sent and the pointer reaches the null terminator, the function ends and returns, ensuring the complete string is successfully transmitted.

- Function UART2 read string: This reads a string from the UART2 receiver into a provided character buffer until it encounters a newline or carriage return, signaling the end of the input string. The function initializes variables data, complete, and num_bytes. With “data” temporarily holds the incoming character, “complete” indicates when to stop reading, and “num_bytes” keeps track of the number of characters stored in the buffer. A while loop continuously checks if complete is 0. Inside this loop, the function checks if

U2STAbits.URXDA is set, indicating that data is available in the receive register U2RXREG. If data is available, it reads the character from U2RXREG into data. The function then checks if data is either a newline (\n) or carriage return (\r). If true, it sets complete to 1, ending the read operation. If data is not a newline or carriage return, the function stores the character in the buffer at the position specified by num_bytes. It then increments num_bytes to move to the next buffer position. To prevent buffer overflow, num_bytes is reset to 0 if it reaches or exceeds maxLength, ensuring the buffer does not exceed its allocated size. Once the loop ends, buffer[num_bytes] is set to '\0' to terminate the string. Finally, _CP0_SET_COUNT(0) is called, resetting the core timer to 0 for timing operations. Then turn on Timer5 by setting T5CONbits.ON = 1, for timing when starting to receive UART data.

- **UART2 ISR Interrupt Handler:** Inside this ISR, the first action is to check if the UART2 receive interrupt flag (IFS1bits.U2RXIF) is set. This flag indicates that data is available in the UART2 receive buffer. If this condition is true, the ISR allocates a buffer array of 20 characters to temporarily store the received message. The function UART2_ReadString is then called, allowing it to read and store a complete message from the UART2 receive buffer. This function reads characters until it encounters a newline or carriage return, storing them in a buffer to create a null-terminated string. After the message is received, it is converted from a string to a floating-point number using the "atof" function. The result is assigned to the variable reference_angle, this is the angle constant 750 from the MATLAB to display as reference and plot in the graph. Finally, IFS1bits.U2RXIF is cleared by setting it to 0, indicating that the ISR has completed handling the current receive event. Clearing the UART2 RX flag at the end.

- **Change Notification Interrupt for Encoder reading:** As implemented from lab 2 and state machine in Figure 21.9 on page 325 of Lynch, the function is for handling change notifications on the encoder inputs connected to the Explorer 16/32 Board. Within this function, the current states of the encoder signals, A and B, are read from the appropriate port registers. The function implements a quadrature decoding state machine that determines the direction of rotation by analyzing the transitions between the previous and current states of the encoder signals. If specific transition patterns are detected, the encoder_count is incremented or decremented to reflect forward or reverse rotation. The previous state values are then updated to maintain the current state for future comparisons. Finally, the current angle is calculated based on the number of encoder counts and converted into degrees. The encoder count is converted into an angular value in degrees by equation $\text{angle} = (\text{encoder_count} * 360.0 / (48.0 * 99.0))$. This takes into account that the encoder provides 48 counts per revolution (CPR) and that the motor has a 99:1 gear ratio. Then under the CNISR, use if statements for if S3 (RD6) or S4 (RD13) is pressed using PORTxbits to trigger the CN interrupt, and set the LED and the mode accordingly to 0 or 1 as using potentiometer or proportional control for later for T2 to manage the direction motor. The interrupt flag clear at the end.

- **Timer2 ISR interrupt:** For exercise E2 (similar for E3 and E4) (mode == S3), this interrupt special register is run by Timer 2 to adjust the motor's speed and direction to minimize the error between a reference angle and the current angle of the motor. The control logic includes direction control using the logic levels of two output pins, F0 and F1, as well as the duty cycle of the PWM signal generated by Output Compare Module 4 (OC4). This system is designed to continuously reduce the angular error while

adjusting the motor's speed based on user input, for this case is the potentiometer for duty cycle. First calculate the error between the reference angle and the current angle of the motor. This error is used to determine whether the motor should be running, and in which direction. For mode as S3 using the potentiometer, if the error is within a tolerance of ± 7.5 degrees, the motor is stopped by setting both F0 and F1 to logic "0". This ensures that no motion occurs when the motor is already close enough to the target angle. If the error is greater than 7.5 degrees, the direction of the motor is set to either increase or decrease the angle. This is done by setting the F1 and F0 pins appropriately: F1 = 0 and F0 = 1 causes the motor to rotate in a direction that decreases the encoder count, while F1 = 1 and F0 = 0 makes the motor rotate in the opposite direction to increase the count. Direction of moving forward or reverse of the motor depends on what positive reference direction is defined at the beginning. These pins control the direction of the motor, ensuring that the system moves toward the reference angle. For controlling the speed of the motor, the OC4RS register is updated in the ISR. In the initial mode (S3), the potentiometer value is read from `int read_potentiometer()` and directly assigned to OC4RS. The potentiometer allows for a straightforward manual adjustment of the motor speed, with the duty cycle increasing as the potentiometer is turned to the right towards 100%, corresponding to faster motor rotation, and decreasing when turned left towards 0%, which slows down or stops the motor. The range of potentiometer values is mapped from 0 to 1023, where 0 represents no motor rotation and 1023 represents full speed. Finally, clear the Timer2 interrupt flag.

- **Timer5 ISR Interrupt:** This is an interrupt special register uses Timer5, running 20 Hz. The first action within the ISR is to define a character array named `data`, which will be used to store the formatted output strings for transmitting over UART. Next, the ISR retrieves the elapsed time since the reference angle was received by utilizing the `_CP0_GET_COUNT()` function, which provides a precise count of clock cycles. This count is then converted to seconds by multiplying by a factor of 25ns, thereby yielding the actual time in seconds. After determining the elapsed time, the ISR formats the output for the elapsed time, reference angle, and current angle into the `data` array using the "sprintf" function. Each formatted string follows the format specifier `%0.4f`, which ensures that the floating-point values are represented with four decimal places for precision. The inclusion of the carriage return (`\r`) and line feed (`\n`) at the end of each string for the receiving MATLAB script. Each of these formatted strings is sent over UART using the `UART2_WriteString()` function. This operation allows for real-time data transmission, enabling the MATLAB script to receive and plot the transient response data effectively. To control the duration of the data transmission, the ISR checks if the elapsed time has reached or exceeded 8 seconds, which corresponds to the collection of 160 samples at the set frequency. If this condition is met, the ISR disables Timer5 by setting `T5CONbits.ON` to 0, effectively stopping further timer interrupts and data transmissions. This step is crucial to prevent the system from collecting unnecessary data beyond the desired limit. Clear the Timer5 interrupt flag by `IFS0bits.T5IF` to 0.

- **Interrupts function, setup_PWM and ADC functions:** First initializes the state of the encoder inputs (A and B) connected to pins RG6 and RG7. The previous state of these pins is stored in the variables `prev_A` and `prev_B`. This function also configures certain LED3, LED4, and LED5 as outputs by setting the corresponding TRISA pins to 0 and turning them off by writing 0 to the LAT register. Similarly, it configures the switches

connected to the microcontroller pins RD6, RD13, and RA7 as inputs. The purpose of this setup is to monitor the states of switches S3, S4, and S5 for different control modes and to control the LEDs as indicators of system states. Further, the function sets up the direction pins (F0 and F1) by configuring them as outputs. Initially, both pins are set to 0, meaning the motor is stopped. After the general setup, the function configures various interrupts. This includes setting up the Change Notification (CN) module for detecting changes in the encoder and switch inputs. The CN interrupt is enabled for pins CN8, CN9 (for the encoder inputs), CN15, and CN19 (for switches S3 and S4). The interrupt priority for these inputs is also set, and the CN interrupt flag is cleared to ensure no unwanted interrupts are pending at the start. Additionally, the INT2 interrupt for switch S5 is configured to trigger on the falling edge, with a high interrupt priority of 7. UART interrupt is set with priority 7 and clear interrupt flag and enable UART2 interrupt. Timer 5, it resets the timer count (TMR5 = 0), sets the period register (PR5) to overflow at 25 Hz based on the system clock frequency (PB_FREQ), and applies a 1:256 prescaler (T5CONbits.TCKPS = 7). The interrupt priority for Timer 5 is set to 3, and the interrupt flag is cleared. The interrupt is enabled (IEC0bits.T5IE = 1), but Timer 5 is initially turned off (T5CONbits.ON = 0) to wait for the read UART function to start communicating. For Timer 2, the timer count is cleared (TMR2 = 0), and the period register (PR2) is set to overflow at a frequency corresponding to 50 MHz with a 1:256 prescaler (T2CONbits.TCKPS = 7). The interrupt priority for Timer 2 is set to 4, the interrupt flag is cleared, and the interrupt is enabled (IEC0bits.T2IE = 1). Timer 2 is then turned on (T2CONbits.ON = 1). The setup_PWM function configures Timer3 and Output Compare (OC4) to generate a PWM signal for controlling the motor speed. Timer3 is set to use an 8:1 prescaler (T3CONbits.TCKPS = 3) and a period value of 1022 at maximum duty cycle (PR3 = 1022), which corresponds to a 10-bit PWM range (0-1023) and timer count PR3 = 0, then timer3 is turn on T3CONbits.ON = 1. This timer will drive the PWM signal used to control the motor's speed. The OC4 module is configured to operate in PWM mode with no fault pin OC4CONbits.OCM = 0b110, using Timer3 as its clock source, and the duty cycle is initially set to 1023, which represents the maximum motor speed. Once the setup is complete, both Timer3 and OC4 are enabled OC4CONbits.ON = 1, allowing the PWM signal to be generated. The setup_ADC function sets up the ADC for reading analog inputs, such as a potentiometer. The function configures the ADC clock source (ADCS = 2), which selects the clock prescaler for the ADC conversion. Pin RA2 is set to analog mode by clearing the corresponding PCFG bit, and the ADC module is turned on by setting the ADON bit in the AD1CON1 register. This enables the ADC for later use in reading analog values.

- `main()` function: The main function begins by declaring two character arrays, `line1` and `line2`, to hold formatted strings for the LCD display. The `lcd_display_driver_initialize()` function prepares the LCD, while `UART2_initialize()` as explained above sets up the UART module for serial communication. The `setup_ADC()` and `setup_PWM()` are called to turn on the ADC and setup T3 and OC4 and turn them on. The `interrupts()` function is then called to configure all interrupts required for timer updates and encoder change notifications. After initialization, the program enters an infinite loop. Within this loop, it formats the reference and current angles into `line1` and `line2` strings using "sprintf", appending the degree symbol (\xDF). The reference angle is displayed on the first line of the LCD, and the current angle is shown on the second line.

This continuous updating allows the LCD to reflect real-time current angle of the encoder and the fixed reference angle of 90 degree from the MATLAB script.

Exercise E3: Proportional Control

- Timer 2 Interrupt: When S4 is triggered in CNISR to turn mode to S3, also trigger T2 running at 50 Hz for condition mode == S3. In the proportional control mode (S4), the error is multiplied by a proportional gain factor to determine the speed of the motor. The resulting output is constrained between 0 and 1023, ensuring that the PWM duty cycle does not exceed the maximum value. The direction is controlled based on the sign of the error, with the motor rotating forward or backward depending on whether the current angle is smaller or larger than the reference angle. When the error is small enough (less than a threshold), the motor stops. The proportional controller ensures that the motor continually adjusts to the reference angle without significant overshoot or steady-state error, though the system does not have a deadband, meaning small errors may still result in continuous adjustments. Clear Timer 2 interrupt flag at the end.

Exercise E4: Proportional-Integral Control

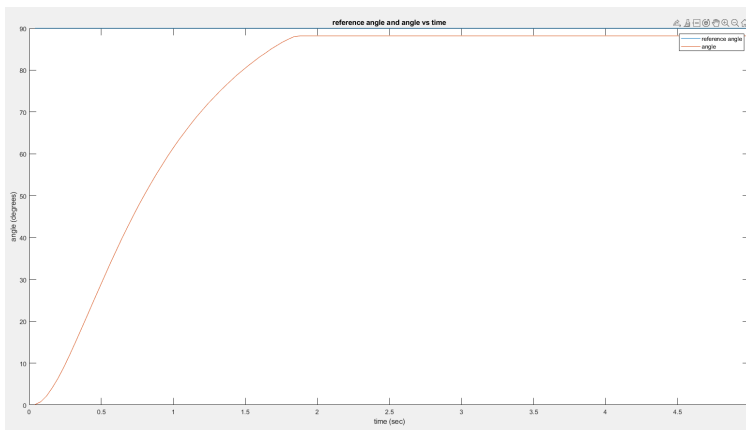
- External 2 Interrupt: S5 is linked to pin 92, since a jumper connects 92 to 19 pin with INT2. Hence when S5 is pressed, External interrupt 2 is triggered, which sets mode to S5 for PI control and turns on LED5. Then clear the interrupt flag for INT2 ISR.
- Timer 2 Interrupt :When S5 is triggered in INT2ISR to turn mode to S5, also trigger T2 running at 50 Hz for condition mode == S5. The proportional-integral control mode (S5) further refines the system by incorporating an integral term. The error is accumulated over time using Euler integration, which sums the error multiplied by the time step (1/50 seconds). This integral term addresses any steady-state error that the proportional controller might not fully eliminate, helping to push the motor towards the target angle even if the error persists over time. The control output in this mode is the sum of the proportional term (scaled by the proportional gain) and the integral term (scaled by the integral gain). This combined output is again clamped to the range of 0 to 1023. The direction is determined by the sign of this combined output, with the motor running in reverse or forward. If the error and integral terms result in zero output, the motor stops, ensuring that the motor does not keep running when it has reached the reference angle, barring any frictional forces in the motor. Clear T2 interrupt flag end.

Section 3: Expected Behavior

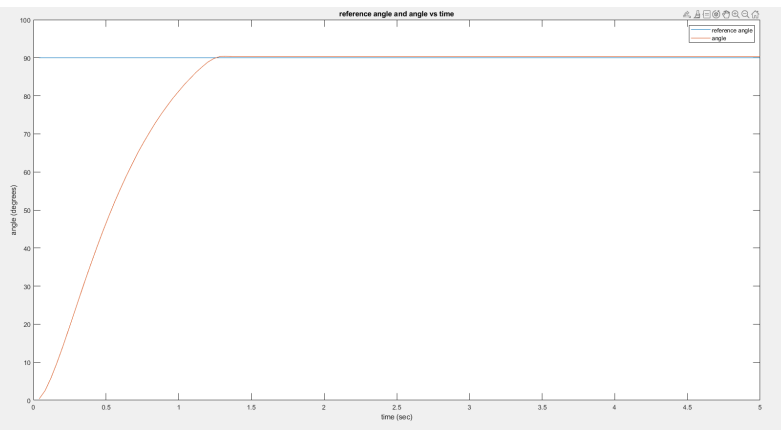
First run the program, then click S3 or S4 or S5 for potentiometer or proportional or proportional-integral control. Then run the matlab, should be able to see the motor turns try to match the desired target 90 degrees. If slide potentiometer all the way to the right for 1023 with 100% the motor turns fast, otherwise if slide to left to 0 the motor will not run. Then, depending on the proportional gain and integral gain value, the plot for current for trying to match the target angle will change, resulting in overdamp, underdamped and critically damp unit step response graphs. This is because a higher K_p increases the corrective action taken by the controller in response to an error, allowing the system to reach its target faster. However, this comes at the cost of increased oscillation, as a higher K_p also tends to make the system more aggressive in correcting errors, often resulting in an overshoot that goes beyond the desired setpoint before settling. This leads to an overdamped response, where the system's response takes longer to stabilize and damp to never reach the desired angle. In contrast, lowering K_p increases the rise time but reduces the oscillation and overshoot,

potentially leading to an underdamped system. The point in between can be found to reach critical damping for rise in short enough time and just reach target angle. For Proportional-Integral Control, the integral gain (K_i), if increased, reduces the steady-state error by continuously integrating the error over time, effectively adding more correction as the system diverges from the setpoint. However, increasing K_i can have the effect of making the system more prone to oscillation and potentially creating an overdamped response. This happens because the integrator accumulates error over time, leading to a cumulative effect that causes the system to overcompensate for the initial error, which can lead to overshooting and oscillation. This can also increase the settling time and cause instability if K_i is too large. On the other hand, decreasing K_i makes the controller less aggressive in correcting the steady-state error, leading to a potentially more stable system. Since the time is 5s, we can not have high K_i to test, for it to stabilize and reduce the error around desired angle.

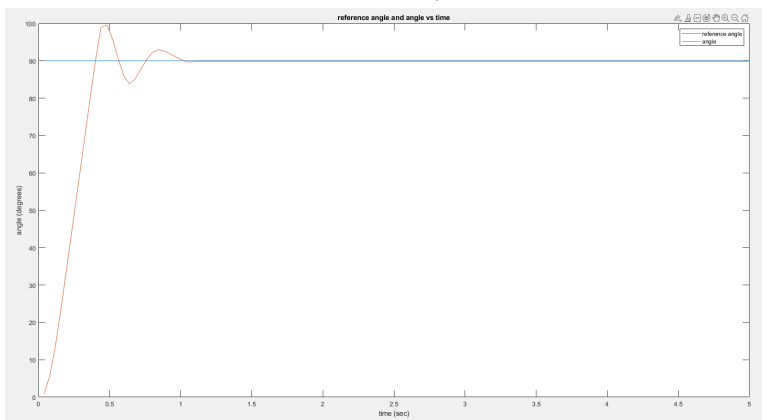
Proportional Gain=5, Integral Gain=0(Overdamp)(P)



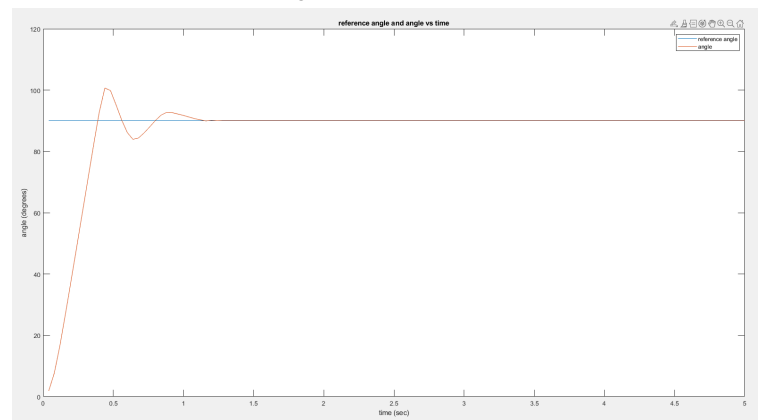
Proportional Gain=6.5, Integral Gain=0(Critical Damp)(P)



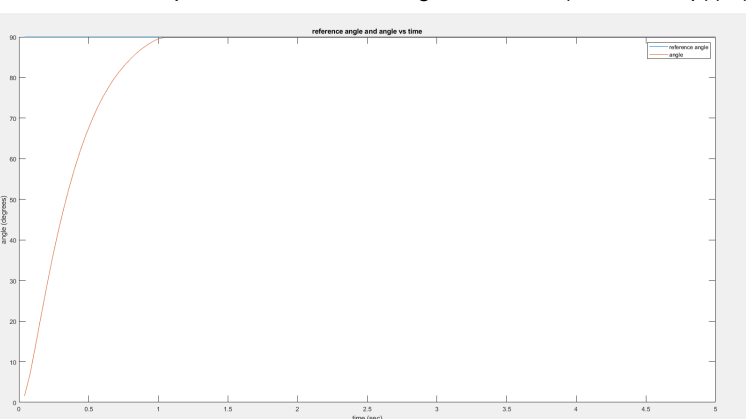
Proportional Gain=100, Integral Gain=0(Underdamp)(P)



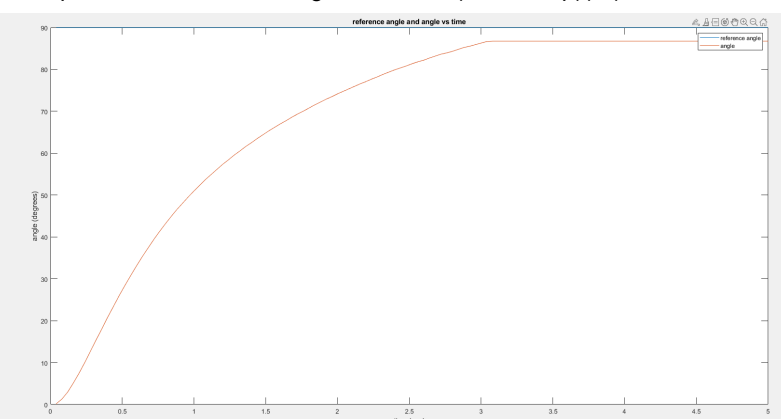
Proportional Gain=80, Integral Gain=2(Underdamp)(PI)



Proportional Gain=10, Integral Gain=0.1(Criticaldamp)(PI)



Proportional Gain=5.5, Integral Gain=0.1(Overdamp)(PI)



Section 4: Code

main.c

```
//SYSCLK and PBCLK operate at 80 MHz
#pragma config POSCMOD = HS
#pragma config FNOSC = PRIPLL
#pragma config FPLLMUL = MUL_20
#pragma config FPLLIDIV = DIV_2
#pragma config FPLLODIV = DIV_1
#pragma config FPBDIV = DIV_1 // Peripheral bus clock divider

#include <xc.h>
#include <stdio.h>
#include <sys/attribs.h>
#include "lcd_display_driver.h"
#include <math.h>

#define BAUDRATE 230400 // desired UART Baud Rate
#define SYS_FREQ 80000000 // System frequency
#define PB_FREQ 80000000 // Peripheral bus frequency
#define U2MODE_VALUE ((PB_FREQ / (16 * BAUDRATE)) - 1) // Baud rate formula
for 16x mode
#define S3 0 // potentiometer
#define S4 1 // proportional control
#define S5 2 // proportional intergral control
volatile int mode = S3; // set mode
volatile float reference_angle = 0.0; // Global variable for the reference angle
volatile float current_angle = 0.0; // Global variable for current angle
volatile float time = 0.0; // Global variable for time measurement
static volatile int prev_A, prev_B; // Previous states of the encoder signals
static volatile int encoder_count = 0; // Global variable for count
static volatile double proportional_gain = 5; // set here
static volatile double integral_gain = 1.0; // set here
static volatile double integral_sum = 0.0; // set here

int read_potentiometer(void) {
    AD1CHSbits.CH0SA = 2; // select AN2
    AD1CON1bits.SAMP = 1;
    unsigned int start = _CP0_GET_COUNT();
    unsigned int current = _CP0_GET_COUNT();
    while ( ( current - start ) < 2000 ) { // sampling time
        current = _CP0_GET_COUNT();
    }
    AD1CON1bits.SAMP = 0; // sampling done, start conversion
    while (!AD1CON1bits.DONE) { // Conversion done
```



```

    ;
}
return ADC1BUF0;
}

void UART2_initialize(void) {
    U2MODEbits.BRGH = 0;    // set M = 16
    U2BRG = U2MODE_VALUE;   // function to set desired baud rate to 230400
    U2MODEbits.PDSEL = 0;   // 8-bit data, no parity
    U2MODEbits.STSEL = 0;   // 1 stop bit
    U2STABits.UTXEN = 1;    // Enable transmission, TX pin
    U2STABits.URXEN = 1;    // Enable reception, RX pin
    U2MODEbits.UEN = 0;     // No configure hardware flow control using RTS and CTS
    U2MODEbits.ON = 1;      // Enable UART2
}

void UART2_WriteString(const char* data) {
    while (*data != '\0') { // Loop until the end of the string
        while (U2STABits.UTXBF); // Wait if buffer is full
        U2TXREG = *data; // Send character
        ++data; // Move to the next character
    }
    return;
}

void UART2_ReadString(char* buffer, int maxLength) {
    char data = 0; // Variable to hold received data
    int complete = 0, num_bytes = 0; // Flags for completion and byte count
    while (!complete) { // Loop until a complete message is received
        if(U2STABits.URXDA){ // Loop until a complete message is received
            data = U2RXREG; // Read the received data

            if((data == '\n') || (data == '\r')) { // Check for end of message
                complete = 1; // Set completion flag
            }
            else {
                buffer[num_bytes] = data; // Store the received character in the buffer
                ++num_bytes; // Increment byte count
                if(num_bytes >= maxLength) { // Check buffer limit
                    num_bytes = 0; // Reset byte count if exceeded
                }
            }
        }
    }
}

```

```

    buffer[num_bytes] = '\0'; // Null-terminate the string
    _CP0_SET_COUNT(0); // Reset the core timer
    T5CONbits.ON = 1; // only when starting receive UART turn timer on
    //T2CONbits.ON = 1;
    //T3CONbits.ON = 1;

    return;
}

void __ISR(_UART_2_VECTOR, IPL7SOFT) U2ISR (void) {

    if(IFS1bits.U2RXIF) {
        // Check if UART2 RX interrupt flag is set
        char buffer[20]; // Buffer to store received message
        UART2_ReadString(buffer, sizeof(buffer)); // Read string into buffer
        reference_angle = atof(buffer); // Convert string to float using atof()
    }
    IFS1bits.U2RXIF = 0; // Clear UART2 RX interrupt flag
}

void __ISR(_CHANGE_NOTICE_VECTOR, IPL6SOFT) CNIS(void) {
    int A = PORTGbits.RG6; // Port register for CN8
    int B = PORTGbits.RG7; // Port register for CN9

    // Implement quadrature decoding state machine
    // Determine the direction of rotation based on transitions
    if ((prev_A == 0 && prev_B == 0 && A == 0 && B == 1) ||
        (prev_A == 0 && prev_B == 1 && A == 1 && B == 1) ||
        (prev_A == 1 && prev_B == 1 && A == 1 && B == 0) ||
        (prev_A == 1 && prev_B == 0 && A == 0 && B == 0)) {
        encoder_count++; // Forward rotation
    }
    else if ((prev_A == 0 && prev_B == 0 && A == 1 && B == 0) ||
        (prev_A == 1 && prev_B == 0 && A == 1 && B == 1) ||
        (prev_A == 1 && prev_B == 1 && A == 0 && B == 1) ||
        (prev_A == 0 && prev_B == 1 && A == 0 && B == 0)) {
        encoder_count--; // Reverse rotation
    }

    // Update the previous state to the current state
    prev_A = A;
    prev_B = B;
    current_angle = (((float)encoder_count * 360)/(48.0 * 99.0)); // Calculate current angle
    based on encoder count
    // Clear the interrupt flag

```

```

    if (PORTDbits.RD6 == 0) { // Check if S3 (CN15) is pressed
        mode = S3;           // Set mode to E2
        LATAbits.LATA0 = 1; // Light up LED 3
        LATAbits.LATA1 = 0; // Turn off LED 4
        LATAbits.LATA2 = 0; // Turn off LED 5
    } else if (PORTDbits.RD13 == 0) { // Check if S4 (CN19) is pressed
        mode = S4;           // Set mode to E3
        LATAbits.LATA0 = 0; // Turn off LED 3
        LATAbits.LATA1 = 1; // Light up LED 4
        LATAbits.LATA2 = 0; // Turn off LED 5
    }

    IFS1bits.CNIF = 0;
    return;
}

void __ISR(_EXTERNAL_2_VECTOR, IPL7SOFT) INT2ISR (void) {
    mode = S5;           // Set mode to E4
    LATAbits.LATA0 = 0; // Turn off LED 3
    LATAbits.LATA1 = 0; // Turn off LED 4
    LATAbits.LATA2 = 1; // Light up LED 5
    // Clear INT2 interrupt flag
    IFS0bits.INT2IF = 0;
    return;
}

void __ISR(_TIMER_2_VECTOR, IPL4SOFT) T2ISR(void) {
    float error = reference_angle - current_angle; // difference with target angle
    if(mode == S3) {
        int potentiometer_value = read_potentiometer();
        OC4RS = potentiometer_value; // set to potentiometer value for duty cycle

        if (abs(error) < 7.5) {
            LATFbits.LATF1 = 0; // Stop motor
            LATFbits.LATF0 = 0;
        } else if (error > 7.5) {
            LATFbits.LATF1 = 0; // Set F1 to 0 for reverse
            LATFbits.LATF0 = 1; // Set F0 to 1
        } else if ((current_angle - reference_angle) > 7.5){
            LATFbits.LATF1 = 1; // Set F1 to 1 for forward
            LATFbits.LATF0 = 0; // Set F0 to 0
        }
    }
    else if(mode == S4) {

```

```

float output = proportional_gain * error; // equation for P control
// Set OC4 compare value to clamped output (0 - 1023)
output = (abs(output) > 1023) ? 1023 : abs(output);
OC4RS = output;
if (current_angle < reference_angle) {
    LATFbits.LATF0 = 1; // Set F0 to 1
    LATFbits.LATF1 = 0; // Set F1 to 0 for reverse
} else if (current_angle > reference_angle){
    LATFbits.LATF0 = 0;
    LATFbits.LATF1 = 1; // other side gogo motor
} else{
    LATFbits.LATF0 = 0;
    LATFbits.LATF1 = 0; // Stop motor
}
}
else if (mode == S5){
    integral_sum += error * (1.0/50); // Euler integration

    float output = proportional_gain * error + integral_gain * integral_sum; // equation
for PI control
    output = (abs(output) > 1023) ? 1023 : abs(output);
    OC4RS = output; // Set PWM duty cycle

    if (output > 0) {
        LATFbits.LATF0 = 1; // Set F0 to 1
        LATFbits.LATF1 = 0; // Set F1 to 0 for reverse
    } else if (output < 0){
        LATFbits.LATF0 = 0;
        LATFbits.LATF1 = 1; // other side gogo motor
    } else{
        LATFbits.LATF0 = 0;
        LATFbits.LATF1 = 0; // Stop motor
    }
}

IFS0bits.T2IF = 0; //Clears the timer flag
}

void __ISR(_TIMER_5_VECTOR, IPL3SOFT) T5ISR(void) {
    char data[50]; // Buffer for formatted output

    unsigned int elapsed = _CP0_GET_COUNT(); // Get elapsed count from the core
timer
    time = elapsed*(25.0e-09); // Convert elapsed count to time in seconds
    // Format output: Time, Reference angle, Current angle

```

```

    sprintf(data, "%0.4f\r\n", time);
    UART2_WriteString(data);

    sprintf(data, "%0.4f\r\n", reference_angle);
    UART2_WriteString(data);

    sprintf(data, "%0.4f\r\n", current_angle);
    UART2_WriteString(data);

    if(time >= 5.0) { // after 125 samples or 5s
        T5CONbits.ON = 0;      // Stop Timer5
    }
    IFS0bits.T5IF = 0; // Clear Timer5 interrupt flag
}

void setup_PWM(void) {

    T3CONbits.TCKPS = 3; // 1:8 prescaler
    PR3 = 1022;          // Period register for 10-bit range
    TMR3 = 0;
    T3CONbits.ON = 1;    // Turn on Timer3

    OC4CONbits.OCM = 0b110; // PWM mode with no fault pin
    OC4CONbits.OCTSEL = 1;  // Use Timer3 for OC4
    OC4RS = 1023;          // Max duty cycle initially
    OC4R = 1023;
    OC4CONbits.ON = 1;     // Turn on OC4
}

void setup_ADC(void) {
    AD1CON3bits.ADCS = 2;
    AD1PCFGbits.PCFG2 = 0;
    AD1CON1bits.ON = 1; // turn ADC on
}

// Setup Change Notification Interrupts for CN8 and CN9
void interrupts(void) {
    // Initialize the previous state of A (CN8) and B (CN9)
    prev_A = PORTGbits.RG6;
    prev_B = PORTGbits.RG7;

    // Configure LEDs as output
    LATAbits.LATA0 = 0; // Turn off LED 3
    LATAbits.LATA1 = 0; // Turn off LED 4
    LATAbits.LATA2 = 0; // Turn off LED 5
    TRISAbits.TRISA0 = 0; // LED 3

```

```

TRISAbits.TRISA1 = 0; // LED 4
TRISAbits.TRISA2 = 0; // LED 5
//TRISA = 0xFF00;

// Configure switches as input
TRISDbits.TRISD6 = 1; // S3 (CN15)
TRISDbits.TRISD13 = 1; // S4 (CN19)
TRISAbits.TRISA7 = 1; // S5 (INT2)

// F0 & F1 for direction of motor
LATFbits.LATF0 = 0;
LATFbits.LATF1 = 0;
TRISFbits.TRISF0 = 0;
TRISFbits.TRISF1 = 0;

DDPCONbits.JTAGEN = 0; // Enable all LEDs
INTCONbits.MVEC = 1; // Enable multi-vector interrupts
__builtin_disable_interrupts(); // Disable interrupts during setup

//Timer 5
TMR5 = 0; // Reset timer count
PR5 = ((PB_FREQ/25)/256)-1; // Set Timer5 to overflow at 25 Hz
T5CONbits.TCKPS = 7; // 1:256 prescaler
IPC5bits.T5IP = 3; // Set Timer5 interrupt priority 6
IFS0bits.T5IF = 0; // Clear Timer5 interrupt flag
IEC0bits.T5IE = 1; // Enable Timer5 interrupt
T5CONbits.ON = 0; // Timer 5, OFF since not receive any signal from UART

//Timer 2
TMR2 = 0;
PR2 = 6249; // ((PB_FREQ/50)/256)-1; //50 MHz at timer 2
T2CONbits.TCKPS = 7; // 1:256 prescaler
IPC2bits.T2IP = 4; // Set Timer5 interrupt priority 6
IFS0bits.T2IF = 0; // Clear Timer5 interrupt flag
IEC0bits.T2IE = 1; // Enable Timer5 interrupt
T2CONbits.ON = 1; // Timer 5, OFF since not receive any signal from UART

//CN Interrupts
CNCONbits.ON = 1; // Enable CN
CNENbits.CNEN8 = 1; // Enable CN for CN8 (pin 10)
CNENbits.CNEN9 = 1; // Enable CN for CN9 (pin 11)
CNENbits.CNEN15 = 1; // Enable CN15 for S3
CNENbits.CNEN19 = 1; // Enable CN19 for S4
IPC6bits.CNIP = 6; // CN Interrupt priority 5
IFS1bits.CNIF = 0; // Clear Change Notification interrupt flag
IEC1bits.CNIE = 1; // Enable CN interrupts

```

```

// Configure INT2 for S5
INTCONbits.INT2EP = 0; // Interrupt on falling edge
IPC2bits.INT2IP = 7; // INT2 interrupt priority 7
IEC0bits.INT2IE = 1; // Enable INT2 interrupt
IFS0bits.INT2IF = 0; // Clear INT2 interrupt flag

//UART Interrupt
U2STAbits.URXISEL = 0x0;
IPC8bits.U2IP = 7; // Interrupt priority UART2 5
IFS1bits.U2RXIF = 0; // Clear UART2 Interrupt flag
IEC1bits.U2RXIE = 1; // Enable UART2 interrupt

__builtin_enable_interrupts(); // re-enable global interrupts
}

int main(void) {
    char line1[16], line2[16]; // Character arrays for storing strings to display on the LCD
    lcd_display_driver_initialize(); // Initialize the LCD display driver
    UART2_initialize(); // Initialize UART
    setup_ADC();
    setup_PWM();
    interrupts(); // Set up all required interrupts
    while (1) {

        // Display reference and current angle on LCD
        int length = sprintf(line1, "Target:%8.2fxDF", reference_angle);
        lcd_display_driver_use_first_line();
        lcd_display_driver_write(line1, length); // Write the reference angle to the LCD

        int length1 = sprintf(line2, "Current:%7.2fxDF", current_angle); // current angle
        from CN interrupt
        lcd_display_driver_use_second_line();
        lcd_display_driver_write(line2, length1); // Write the current angle to the LCD
    }
    return 0;
}

```

lcd_display_driver.c

```

#include<xc.h>
#include"lcd_display_driver.h"
#include<string.h>

// This function uses a for loop to create a delay.
// Delays execution for a specified number of steps.

```

```

void lcd_display_driver_delay(int steps){
    int i = 0;
    for( i = 0; i < steps; i++ ){
        return;
    }
    // Set the toggles the enable bit to perform a display operation
    void lcd_display_driver_enable(){
        TRISDbits.TRISD4 = 0; // Set E as RD4 as output for the enable signal
        // fill in line to raise E from 0 to 1
        LATDbits.LATD4=1;
        lcd_display_driver_delay( 10000 ); // // Delay to allow the LCD to process the enable
        signal as initialization diagram
        // fill in line to lower E from 1 to 0
        LATDbits.LATD4=0;
        lcd_display_driver_delay( 10000 );
        return;
    }

    // Perform the sequence of initialization operations
    // Bring up display for it operates in a mode with 5x7 dots, dua line, no cursor
    void lcd_display_driver_initialize(){
        //Function Set
        TRISBbits.TRISB15=0; // RS port bit as output
        TRISDbits.TRISD5=0; // R/W port bit as output
        TRISE=0xFF00; // // DB0-7 port bits as output

        // These from the Novatek datasheet for I/O ports R/W, RS, DB0-7
        LATBbits.LATB15=0; // RS sets to 0
        LATDbits.LATD5=0; // R/W sets to 0
        LATE=0b00111000; // DB0-7 sets to 00111000
        lcd_display_driver_enable();

        //Display ON/OFF
        LATBbits.LATB15=0; // RS sets to 0
        LATDbits.LATD5=0; // R/W sets to 0
        LATE=0b00001100; // DB0-7 sets to 00001100
        lcd_display_driver_enable();

        //Clear
        lcd_display_driver_clear();

        //Entry Mode Set
        LATBbits.LATB15=0; // RS sets to 0
        LATDbits.LATD5=0; // R/W sets to 0
        LATE=0b00000110; // DB0-7 sets to 00000110
    }
}

```



```

    lcd_display_driver_enable();
}

// Clear the display of all characters
void lcd_display_driver_clear(){
    // From Novatek datasheet
    LATBbits.LATB15=0; // RS sets to 0
    LATDbits.LATD5=0; // R/W sets to 0
    LATE=0b00000001; // DB0-7 sets to 00000001
    lcd_display_driver_enable();
}

// Writes a string of characters to the LCD, with the given length.
void lcd_display_driver_write(char* data, int length){
    LATBbits.LATB15=1; // RS to 1 as data mode
    LATDbits.LATD5=0; // R/W to 0 as write mode
    int i = 0;
    for(i=0; i<length; i++) {
        LATE=data[i]; // Write each character to the data register
        lcd_display_driver_enable();
    }
}

// Sets the LCD cursor to the start of the first line (line 0) of the display.
void lcd_display_driver_use_first_line(void){
    LATBbits.LATB15=0; // RS set to 0 for command mode
    LATDbits.LATD5=0; // R/W set to 0 for write mode
    LATE = 0x80; // Offset 0 start as first line
    lcd_display_driver_enable();
}

void lcd_display_driver_use_second_line(void){
    LATBbits.LATB15=0; // RS set to 0 for command mode
    LATDbits.LATD5=0; // R/W set to 0 for write mode
    LATE=0xC0; // Offset 0
    lcd_display_driver_enable();
}

```

lcd_display_driver.h

```

#ifndef _LCD_DISPLAY_DRIVER_H /* Guard against multiple inclusion */
#define _LCD_DISPLAY_DRIVER_H

// Function to introduce a delay for the LCD display.

```

```
// 'steps' parameter determines how long the delay lasts.
void lcd_display_driver_delay(int steps);

// Function to toggle the enable pin of the LCD, which triggers a display operation.
// Signal LCD to execute a command or write data.
void lcd_display_driver_enable();

// Function to initialize the LCD display.
// Sets the display mode (e.g., character size, line mode)
void lcd_display_driver_initialize();

// Function to clear the LCD display of all characters.
// Reset the display to a blank state.
void lcd_display_driver_clear();

// Function to write data to the LCD display.
// 'data' is the character array to be written, and 'length' is the number of characters to
// write.
void lcd_display_driver_write(char* data, int length);

// Function to set the LCD cursor to the first line.
// Future writes will begin at the start of the first line.
void lcd_display_driver_use_first_line(void);

// Function to set the LCD cursor to the second line.
// Future writes will begin at the start of the second line.
void lcd_display_driver_use_second_line(void);

#endif /* _LCD_DISPLAY_DRIVER_H */
```